

Date	15 March 2026	Maximum Marks	4 Marks
Team ID	6aba389eb2817f097463f0f8	Project Name	LegalEase: AI-Powered Legal Document Generator

SOLUTION REQUIREMENTS DOCUMENT

Project Description & Scenarios

LegalEase leverages state-of-the-art Generative AI to simplify the creation of legal documents by providing customizable, accurate, and editable templates for a wide range of use cases. Users can generate employment contracts, lease agreements, NDAs, and more—all tailored to their specific inputs like involved parties, effective dates, and key terms.

Built with FastAPI and integrated with a powerful AI core (Google Gemini 1.5 Pro), LegalEase ensures users receive professional-grade legal documents that maintain formatting standards, include custom branding (logos and fonts), and support features like editable previews and automatic term tables. With secure handling of sensitive user data and seamless document export options (.PDF, .DOCX, .TXT), the platform bridges the gap between accessibility and legal professionalism.

Scenarios:

- **Scenario 1:** A startup founder enters details about a new hire and selects "Employment Contract" as the document type. LegalEase generates a comprehensive agreement with clauses for roles, responsibilities, compensation, and confidentiality. The user customizes the final wording, adds their company logo, and downloads the document as a branded PDF.
- **Scenario 2:** A freelancer needs a Non-Disclosure Agreement for a potential client. By specifying the parties, scope of confidentiality, and effective date, they instantly receive a structured and legally sound NDA.
- **Scenario 3:** A landlord uses LegalEase to draft a residential lease agreement. By inputting the property address, tenant details, and lease terms, the platform creates a formatted document with editable sections and a downloadable version in .DOCX format.

Step 1: Functional Requirements (FR)

Functional Requirements define specific behaviors, functions, or features of the system (What the system should do).

S.No	Requirement Category	Requirement Description	Priority
1	Authentication & Authorization levels	Users access the frontend interface securely. Environment key configurations (.env containing Gemini API Key) protect backend AI execution. User role access levels dictate document creation and modification rights.	High
2	External interfaces	Integration with Google Generative AI API (Gemini 1.5 Pro model) via `google-generativeai` SDK. FastAPI backend communicates seamlessly with Streamlit frontend via REST endpoints. Export rendering modules connect with document libraries (`python-docx`, `fpdf`).	High
3	Transactions processing	Processes document generation requests accepting `document_type`, `parties`, `terms`, and `dates` via JSON payloads. Handles asynchronous POST API processing to deliver dynamic AI generated content in real-time.	High

4	Reporting & Exports	Generates styled dynamic document previews with dark-theme HTML cards. Provides multi-format export/download capabilities including .TXT	High
S.No	Requirement Category	Requirement Description	Priority
		(Plain Text), .DOCX (Formatted Word Document with logo, headings, table), and .PDF (Branded PDF with logo and footer).	
5	Business rules, etc.	Sanitizes generated AI text (`sanitize_text`), auto-parses semicolon-separated input clauses into structured terms tables, enforces formal legal document formatting with headers, footers, and signatures. Provides inline live text editing prior to download.	Medium
6	Compliance to laws or regulations	Adheres to data privacy standards during prompt processing. Generated templates follow standard contract structure guidelines ensuring confidentiality and legal wording accuracy.	Medium
7	Other	Interactive frontend with Streamlit displaying company branding logos, dynamic responsive input forms, and status feedback banners ("Document Generated Successfully").	Low

Step 2: Non-Functional Requirements (NFR)

Non-Functional Requirements specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should be).

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Fast execution of document prompts using Gemini 1.5 Pro model and FastAPI asynchronous routing.	API response time < 3 seconds; Frontend page load < 2 seconds.
2	Scalability	Modular microservice-ready backend (FastAPI Uvicorn) capable of handling concurrent legal drafting requests.	Supports up to 1,000 concurrent document generation requests.
3	Security & Data Privacy	Secure API key management via `.env` file; secure handling of sensitive user document details.	Zero hardcoded secrets; TLS/HTTPS secure endpoint transmission.
4	Reliability & Availability	High uptime for FastAPI backend server and Streamlit frontend UI.	99.5% system uptime and robust exception handling for API timeouts.
5	Usability & Accessibility	Intuitive, clean dark-themed user interface requiring no prior legal or technical training.	Complies with modern web UI/UX standards; seamless multi-format export buttons.
6	Maintainability & Portability	Modular Python codebase structured across `ai_core`, `legalEaseAPI`, and `frontend`. Environment dependencies managed via `requirements.txt`.	Deployable on Render, Railway, Streamlit Community Cloud, or Docker containers.

Technical Implementation & Workflow Milestones

System Architecture & Tech Stack

- **Frontend:** Streamlit (`app.py`) for user interaction, input collection, styled preview, and multi-format downloads.
- **Backend:** FastAPI (`main.py`, `routes.py`) running on Uvicorn for handling RESTful POST requests (`/generate`).

- **AI Core:** Google Gemini 1.5 Pro integrated inside ``ai_core/gemini_generator.py`` for structured legal drafting.
- **Export Utilities:** ``python-docx`` for Word formatting, ``fpdf`` / ``Pillow`` for PDF creation, and Python standard I/O for ``txt``.

Development Milestones Summary:

- **Milestone 1: Model Selection & Architecture** — Selected ``gemini-1.5-pro`` for its 1M token context window and structured prompt capabilities. Established project directory structure.
- **Milestone 2: Core Functionalities Development** — Developed backend prompt generator, sanitize utilities, HTML/PDF/DOCX formatting scripts, and Streamlit state management.
- **Milestone 3: API Logic Integration** — Created FastAPI app in ``main.py``, configured Pydantic ``DocumentRequest`` schema, and implemented ``/generate`` POST endpoint in ``routes.py``.
- **Milestone 4: Frontend Development** — Built centered multi-column layout in Streamlit, styled HTML preview card, inline text editor, and download triggers.
- **Milestone 5: Deployment & Verification** — Configured virtual environment, environment key management (``.env``), verified end-to-end local deployment (``uvicorn`` + ``streamlit run``), and performed complete functional testing.

Conclusion

LegalEase represents a transformative step in bridging the accessibility gap in legal services through the power of artificial intelligence. By integrating advanced natural language processing (NLP) models with user-centric design, LegalEase empowers individuals and business founders to navigate legal content with confidence, promoting inclusivity, clarity, and efficiency.